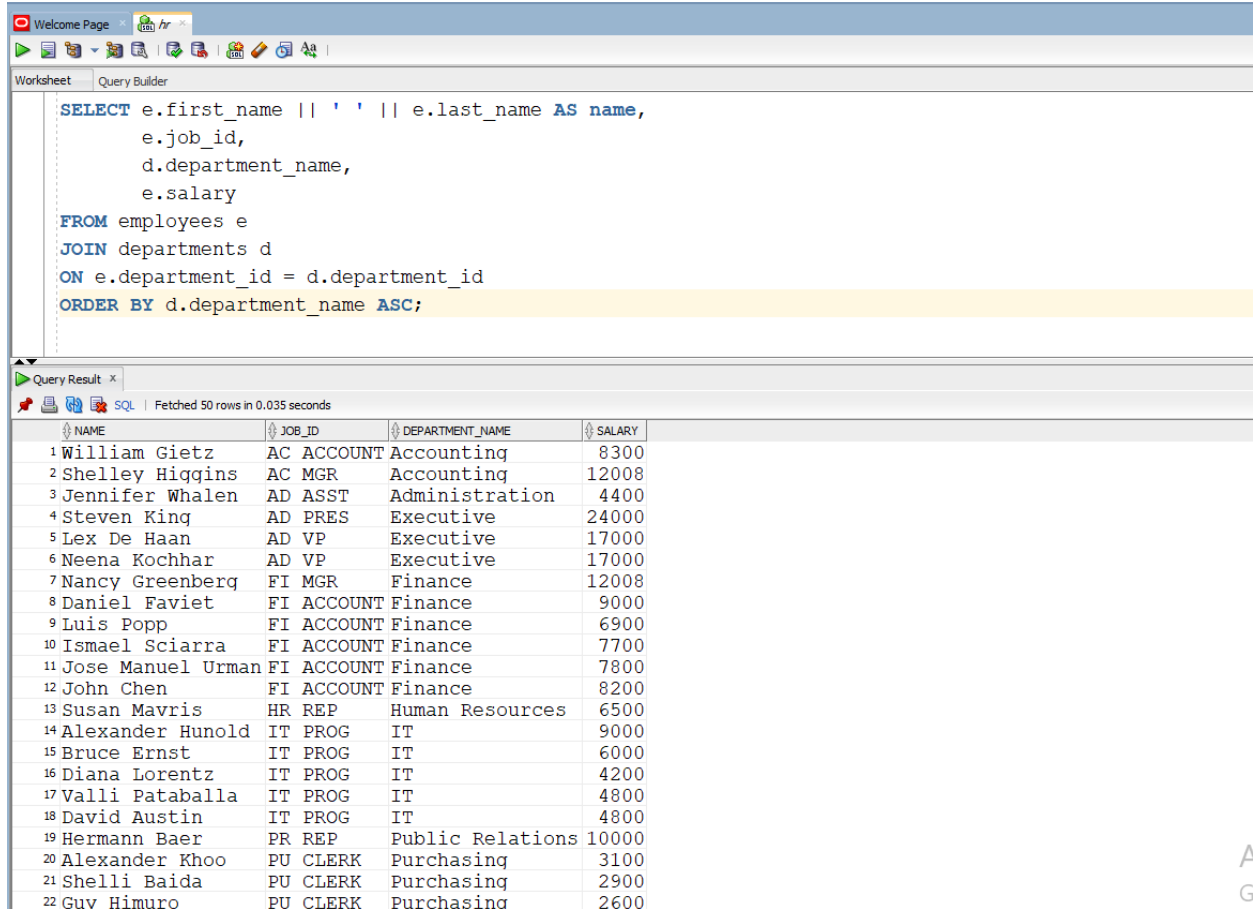


DBS LAB 7

IN LAB TASK

TASK 1:



The screenshot shows a database query tool interface. The top part is a 'Query Builder' window with a SQL query. The bottom part is a 'Query Result' window showing the results of the query. The query is a SELECT statement that joins the 'employees' table with the 'departments' table. The results are displayed in a table with 5 columns: NAME, JOB_ID, DEPARTMENT_NAME, and SALARY. The results are sorted by DEPARTMENT_NAME in ascending order.

```
SELECT e.first_name || ' ' || e.last_name AS name,
       e.job_id,
       d.department_name,
       e.salary
FROM employees e
JOIN departments d
ON e.department_id = d.department_id
ORDER BY d.department_name ASC;
```

	NAME	JOB_ID	DEPARTMENT_NAME	SALARY
1	William Gietz	AC ACCOUNT	Accounting	8300
2	Shelley Higgins	AC MGR	Accounting	12008
3	Jennifer Whalen	AD ASST	Administration	4400
4	Steven King	AD PRES	Executive	24000
5	Lex De Haan	AD VP	Executive	17000
6	Neena Kochhar	AD VP	Executive	17000
7	Nancy Greenberg	FI MGR	Finance	12008
8	Daniel Faviat	FI ACCOUNT	Finance	9000
9	Luis Popp	FI ACCOUNT	Finance	6900
10	Ismael Sciarra	FI ACCOUNT	Finance	7700
11	Jose Manuel Urman	FI ACCOUNT	Finance	7800
12	John Chen	FI ACCOUNT	Finance	8200
13	Susan Mavris	HR REP	Human Resources	6500
14	Alexander Hunold	IT PROG	IT	9000
15	Bruce Ernst	IT PROG	IT	6000
16	Diana Lorentz	IT PROG	IT	4200
17	Valli Pataballa	IT PROG	IT	4800
18	David Austin	IT PROG	IT	4800
19	Hermann Baer	PR REP	Public Relations	10000
20	Alexander Khoo	PU CLERK	Purchasing	3100
21	Shelli Baida	PU CLERK	Purchasing	2900
22	Guy Himuro	PU CLERK	Purchasing	2600

A
G

TASK 2:

The screenshot shows a database query tool interface. The top bar includes a 'Welcome Page' tab and a toolbar with various icons. Below the toolbar, the 'Worksheet' tab is active, displaying a SQL query in the 'Query Builder' section. The query is as follows:

```
SELECT department_id, COUNT(*) AS emp_count
FROM employees
GROUP BY department_id
HAVING COUNT(*) >= 2;
```

Below the query, the 'Query Result' tab is active, showing the results of the query. The results are displayed in a table with two columns: 'DEPARTMENT_ID' and 'EMP_COUNT'. The table contains 7 rows of data.

	DEPARTMENT_ID	EMP_COUNT
1	100	6
2	30	6
3	90	3
4	110	2
5	50	45
6	80	34
7	60	5

TASK 3:

The screenshot shows a database query tool interface. At the top, there's a 'Welcome Page' tab and a 'Query Builder' tab. The 'Query Builder' tab is active, displaying an SQL query. Below the query, there's a 'Query Result' tab showing the results of the query. The results are displayed in a table with 5 columns: NAME, JOB_ID, ANNUAL_SALARY, DEPARTMENT_NAME, and CITY. The table contains 18 rows of data, numbered 1 through 18. The query filters for employees with an annual salary greater than or equal to 60,000 and who are not analysts.

```
SELECT e.first_name || ' ' || e.last_name AS name,
       e.job_id,
       e.salary*12 AS annual_salary,
       d.department_name,
       l.city
FROM employees e
JOIN departments d
ON e.department_id = d.department_id
JOIN locations l
ON d.location_id = l.location_id
WHERE e.salary*12 >= 60000
AND e.job_id <> 'ANALYST';
```

	NAME	JOB_ID	ANNUAL_SALARY	DEPARTMENT_NAME	CITY
1	Bruce Ernst	IT PROG	72000	IT	Southlake
2	Alexander Hunold	IT PROG	108000	IT	Southlake
3	Kevin Mourgos	ST MAN	69600	Shipping	South San Francisco
4	Matthew Weiss	ST MAN	96000	Shipping	South San Francisco
5	Adam Fripp	ST MAN	98400	Shipping	South San Francisco
6	Payam Kaufling	ST MAN	94800	Shipping	South San Francisco
7	Shanta Vollman	ST MAN	78000	Shipping	South San Francisco
8	William Gietz	AC ACCOUNT	99600	Accounting	Seattle
9	Shelley Higgins	AC MGR	144096	Accounting	Seattle
10	Luis Popp	FI ACCOUNT	82800	Finance	Seattle
11	Nancy Greenberg	FI MGR	144096	Finance	Seattle
12	Daniel Faviet	FI ACCOUNT	108000	Finance	Seattle
13	John Chen	FI ACCOUNT	98400	Finance	Seattle
14	Ismael Sciarra	FI ACCOUNT	92400	Finance	Seattle
15	Jose Manuel Urman	FI ACCOUNT	93600	Finance	Seattle
16	Neena Kochhar	AD VP	204000	Executive	Seattle
17	Steven King	AD PRES	288000	Executive	Seattle
18	Lex De Haan	AD VP	204000	Executive	Seattle

TASK 4:

The screenshot shows the SQL Developer interface with a query in the Query Builder. The query is designed to select all employees who do not have a manager assigned to them. The query text is as follows:

```
SELECT *
FROM employees
WHERE employee_id IN (
    SELECT manager_id
    FROM employees
    WHERE manager_id IS NOT NULL
);
```

Below the query, the Query Result pane displays the results of the query. It shows 17 rows of data, representing employees who do not have a manager. The columns displayed are: EMPLOYEE_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, HIRE_DATE, JOB_ID, SALARY, COMMISSION_PCT, MANAGER_ID, and DEPARTMENT_ID.

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID	
1	100	Steven	King	SKING	515.123.4567	17-JUN-03	AD PRES	24000	(null)	(null)	90
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21-SEP-05	AD VP	17000	(null)	100	90
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13-JAN-01	AD VP	17000	(null)	100	90
4	103	Alexander	Hunold	AHUNOLD	590.423.4567	03-JAN-06	IT PROG	9000	(null)	102	60
5	108	Nancy	Greenberg	NGREENBE	515.124.4569	17-AUG-02	FI MGR	12008	(null)	101	100
6	114	Den	Raphaely	DRAPHEAL	515.127.4561	07-DEC-02	PU MAN	11000	(null)	100	30
7	120	Matthew	Weiss	MWEISS	650.123.1234	18-JUL-04	ST MAN	8000	(null)	100	50
8	121	Adam	Frapp	AFRIPP	650.123.2234	10-APR-05	ST MAN	8200	(null)	100	50
9	122	Payam	Kaufling	PKAUFLIN	650.123.3234	01-MAY-03	ST MAN	7900	(null)	100	50
10	123	Shanta	Vollman	SVOLLMAN	650.123.4234	10-OCT-05	ST MAN	6500	(null)	100	50
11	124	Kevin	Mourgos	KMOURGOS	650.123.5234	16-NOV-07	ST MAN	5800	(null)	100	50
12	145	John	Russell	JRUSSEL	011.44.1344.429268	01-OCT-04	SA MAN	14000	0.4	100	80
13	146	Karen	Partners	KPARTNER	011.44.1344.467268	05-JAN-05	SA MAN	13500	0.3	100	80
14	147	Alberto	Errazuriz	AERRAZUR	011.44.1344.429278	10-MAR-05	SA MAN	12000	0.3	100	80
15	148	Gerald	Cambraut	GCAMBRAU	011.44.1344.619268	15-OCT-07	SA MAN	11000	0.3	100	80
16	149	Eleni	Zlotkey	EZLOTKEY	011.44.1344.429018	29-JAN-08	SA MAN	10500	0.2	100	80
17	205	Shelley	Higgins	SHIGGINS	515.123.8080	07-JUN-02	AC MGR	12008	(null)	101	110

TASK 5:

The screenshot shows the SQL Developer interface with a query in the Query Builder. The query is designed to select all departments that do not have any employees assigned to them. The query text is as follows:

```
SELECT d.department_id, d.department_name
FROM departments d
LEFT JOIN employees e
ON d.department_id = e.department_id
WHERE e.employee_id IS NULL;
```

Below the query, the Query Result pane displays the results of the query. It shows 16 rows of data, representing departments that do not have any employees. The columns displayed are: DEPARTMENT_ID and DEPARTMENT_NAME.

DEPARTMENT_ID	DEPARTMENT_NAME
1	120 Treasury
2	130 Corporate Tax
3	140 Control And Credit
4	150 Shareholder Services
5	160 Benefits
6	170 Manufacturing
7	180 Construction
8	190 Contracting
9	200 Operations
10	210 IT Support
11	220 NOC
12	230 IT Helpdesk
13	240 Government Sales
14	250 Retail Sales
15	260 Recruiting
16	270 Payroll

TASK 6:

The screenshot shows a SQL query editor window with a 'Query Builder' tab. The query is as follows:

```
SELECT e.first_name || ' ' || e.last_name AS name,  
       e.salary,  
       d.department_name  
FROM employees e  
LEFT JOIN departments d  
ON e.department_id = d.department_id;
```

Below the query editor, the 'Query Result' window displays the results of the query. It shows 26 rows of data, with columns for NAME, SALARY, and DEPARTMENT_NAME. The results are as follows:

	NAME	SALARY	DEPARTMENT_NAME
1	Jennifer Whalen	4400	Administration
2	Karen Colmenares	2500	Purchasing
3	Guy Himuro	2600	Purchasing
4	Sigal Tobias	2800	Purchasing
5	Shelli Baida	2900	Purchasing
6	Alexander Khoo	3100	Purchasing
7	Den Raphaely	11000	Purchasing
8	Susan Mavris	6500	Human Resources
9	Douglas Grant	2600	Shipping
10	Donald OConnell	2600	Shipping
11	Kevin Feeney	3000	Shipping
12	Alana Walsh	3100	Shipping
13	Vance Jones	2800	Shipping
14	Samuel McCain	3200	Shipping
15	Britney Everett	3900	Shipping
16	Sarah Bell	4000	Shipping
17	Randall Perkins	2500	Shipping
18	Timothy Gates	2900	Shipping
19	Jennifer Dilly	3600	Shipping
20	Kelly Chung	3800	Shipping
21	Anthony Cabrio	3000	Shipping
22	Julia Dellinger	3400	Shipping
23	Alexis Bull	4100	Shipping
24	Nandita Sarchand	4200	Shipping
25	Girard Geoni	2800	Shipping
26	Martha Sullivan	2500	Shipping

TASK 7:

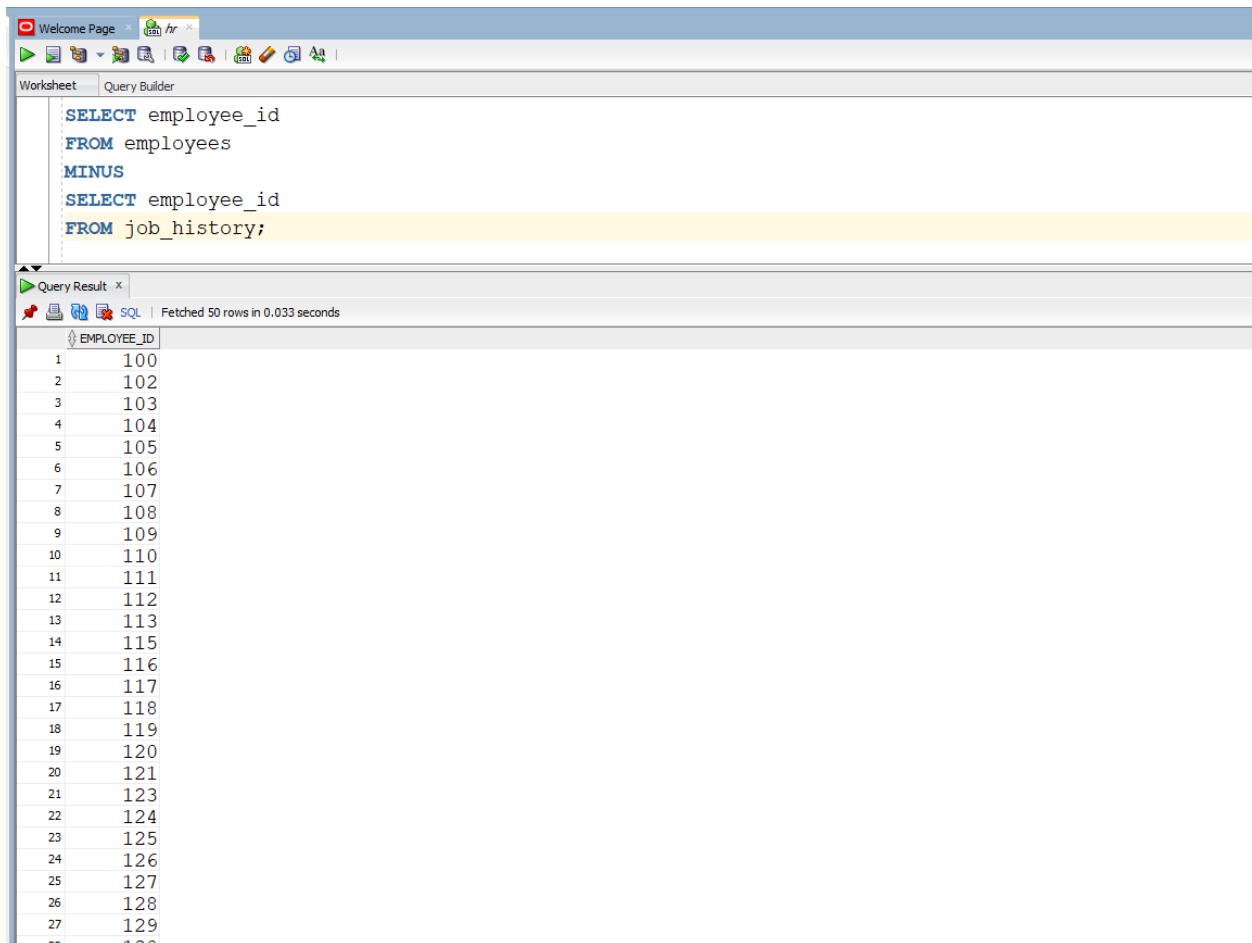
The screenshot displays the Oracle SQL Developer interface. The top window is titled 'Welcome Page' and 'hr'. Below the toolbar, the 'Query Builder' tab is active, showing the following SQL query:

```
SELECT e.first_name || ' ' || e.last_name AS name,  
       e.job_id,  
       d.department_name,  
       l.city  
FROM employees e  
JOIN departments d  
ON e.department_id = d.department_id  
JOIN locations l  
ON d.location_id = l.location_id  
WHERE l.state_province IS NULL;
```

The 'Query Result' window at the bottom shows the execution results. It indicates 'All Rows Fetched: 1 in 0.092 seconds'. The results are displayed in a table with the following columns: NAME, JOB_ID, DEPARTMENT_NAME, and CITY.

NAME	JOB_ID	DEPARTMENT_NAME	CITY
1 Susan Mavris	HR	REP Human Resources	London

TASK 8:



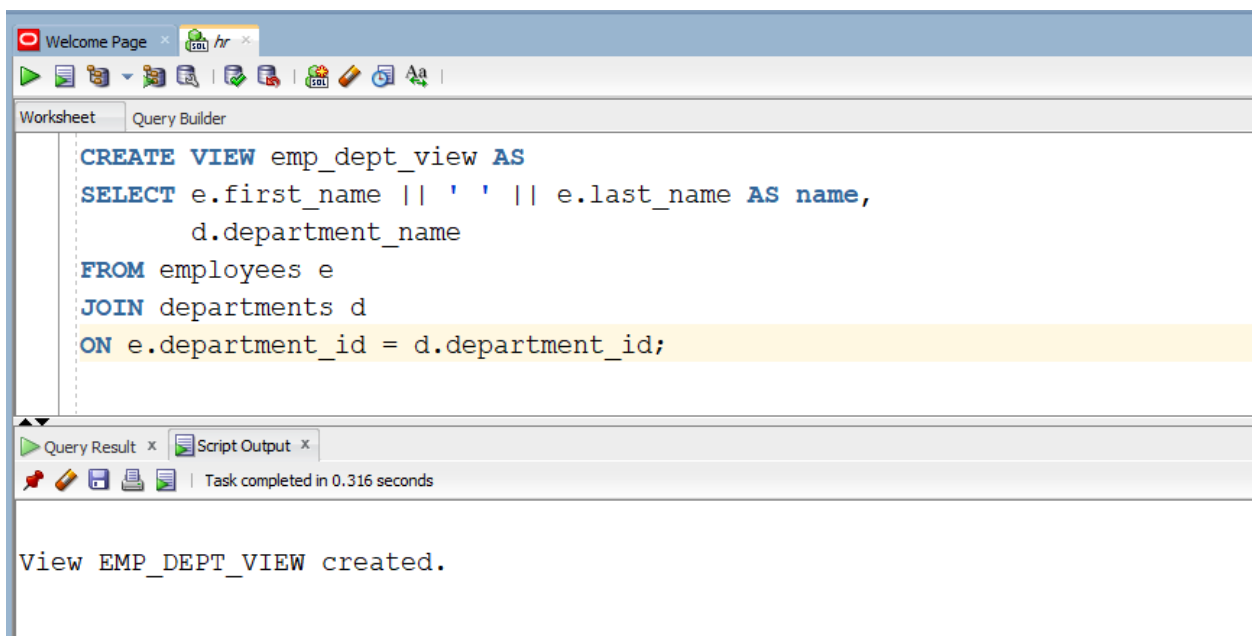
The screenshot shows the SQL Developer interface. The top toolbar includes icons for Welcome Page, hr, and various development tools. The main window is divided into a Worksheet and a Query Builder. The Worksheet contains the following SQL query:

```
SELECT employee_id
FROM employees
MINUS
SELECT employee_id
FROM job_history;
```

Below the query, the Query Result tab is active, showing the results of the query. The status bar indicates "Fetched 50 rows in 0.033 seconds". The results are displayed in a table with the following data:

EMPLOYEE_ID
100
102
103
104
105
106
107
108
109
110
111
112
113
115
116
117
118
119
120
121
123
124
125
126
127
128
129

TASK 9:



The screenshot shows the SQL Developer interface. The top toolbar includes icons for Welcome Page, hr, and various development tools. The main window is divided into a Worksheet and a Query Builder. The Worksheet contains the following SQL query:

```
CREATE VIEW emp_dept_view AS
SELECT e.first_name || ' ' || e.last_name AS name,
       d.department_name
FROM employees e
JOIN departments d
ON e.department_id = d.department_id;
```

Below the query, the Query Result tab is active, showing the results of the query. The status bar indicates "Task completed in 0.316 seconds". The results are displayed in a table with the following data:

NAME	DEPARTMENT_NAME
100	ACCOUNTING
102	ACCOUNTING
103	ACCOUNTING
104	ACCOUNTING
105	ACCOUNTING
106	ACCOUNTING
107	ACCOUNTING
108	ACCOUNTING
109	ACCOUNTING
110	ACCOUNTING
111	ACCOUNTING
112	ACCOUNTING
113	ACCOUNTING
115	ACCOUNTING
116	ACCOUNTING
117	ACCOUNTING
118	ACCOUNTING
119	ACCOUNTING
120	ACCOUNTING
121	ACCOUNTING
123	ACCOUNTING
124	ACCOUNTING
125	ACCOUNTING
126	ACCOUNTING
127	ACCOUNTING
128	ACCOUNTING
129	ACCOUNTING

View EMP_DEPT_VIEW created.

TASK 10:

The screenshot shows the SQL Developer interface with the 'Query Builder' tab active. The SQL editor contains the following code:

```
CREATE OR REPLACE VIEW emp_dept_view AS
SELECT e.first_name || ' ' || e.last_name AS name,
       d.department_name,
       e.salary
FROM employees e
JOIN departments d
ON e.department_id = d.department_id;
```

Below the editor, the 'Query Result' tab shows the message: 'View EMP_DEPT_VIEW created.'.

POST LAB TASK

TASK 1:

The screenshot shows the SQL Developer interface with the 'Query Result' tab active. The SQL editor contains the following code:

```
SELECT DISTINCT e.employee_id,
                e.first_name,
                e.last_name
FROM employees e
JOIN job_history j
ON e.employee_id = j.employee_id;
```

Below the editor, the 'Query Result' tab displays the following data:

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME
1	101	Neena	Kochhar
2	200	Jennifer	Whalen
3	114	Den	Raphaely
4	122	Payam	Kaufling
5	176	Jonathon	Taylor

TASK 2:

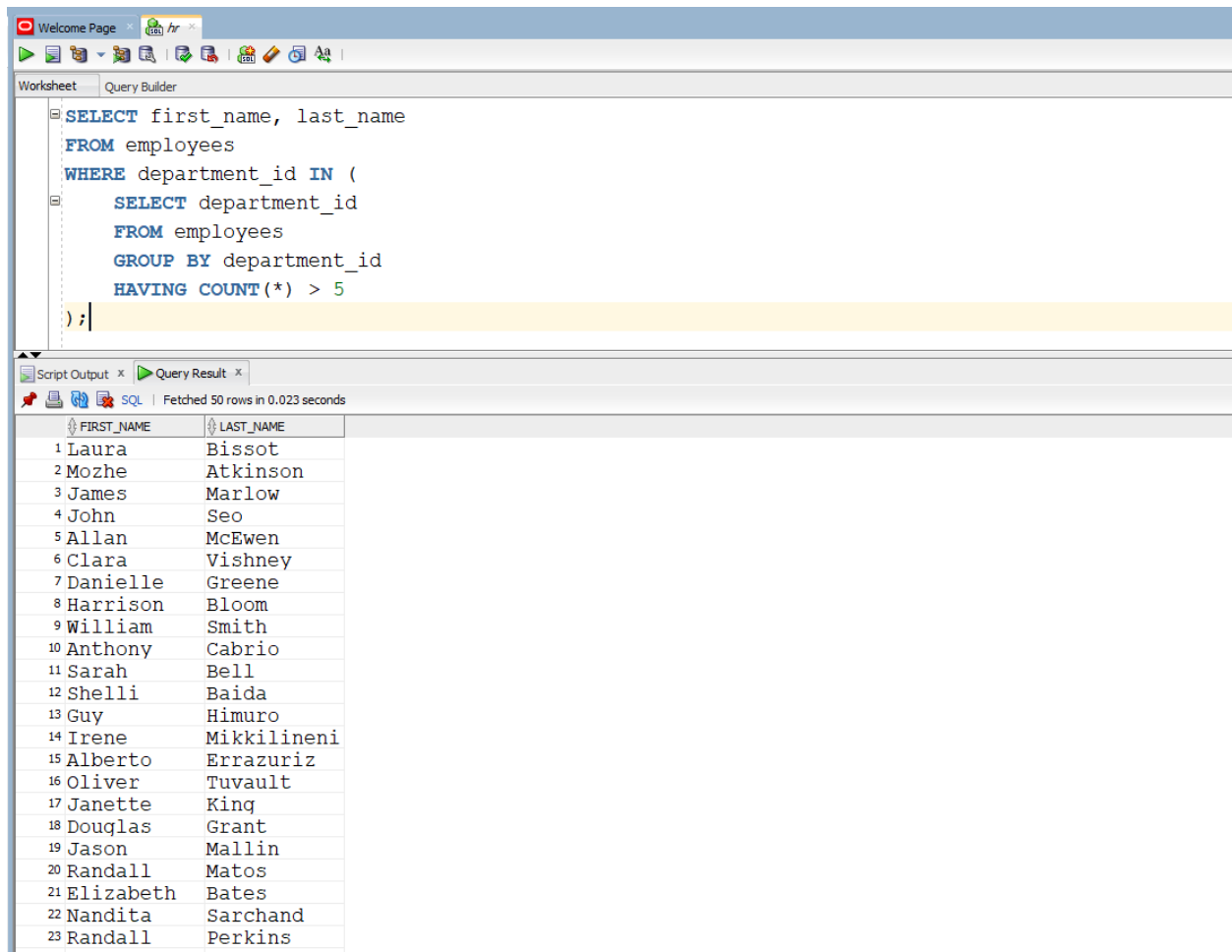
The screenshot shows a SQL query editor with a 'Query Builder' tab. The query is a self-join on the 'employees' table, selecting the first and last names of employees who are managed by others in the same department.

```
SELECT e.first_name, e.last_name
FROM employees e
JOIN employees m
ON e.manager_id = m.employee_id
WHERE e.department_id = m.department_id;
```

Below the query, the 'Query Result' tab displays 50 rows of data. The results are as follows:

	FIRST_NAME	LAST_NAME
1	Lex	De Haan
2	Neena	Kochhar
3	Diana	Lorentz
4	Valli	Pataballa
5	David	Austin
6	Bruce	Ernst
7	Luis	Popp
8	Jose Manuel	Urman
9	Ismael	Sciarra
10	John	Chen
11	Daniel	Faviet
12	Karen	Colmenares
13	Guy	Himuro
14	Sigal	Tobias
15	Shelli	Baida
16	Alexander	Khoo
17	Girard	Geoni
18	Martha	Sullivan
19	Jean	Fleaur
20	Winston	Taylor
21	Steven	Markle
22	James	Landry
23	Irene	Mikkilineni
24	Julia	Nayer
25	Anthony	Cabrio
26	Julia	Dellinger

TASK 3:

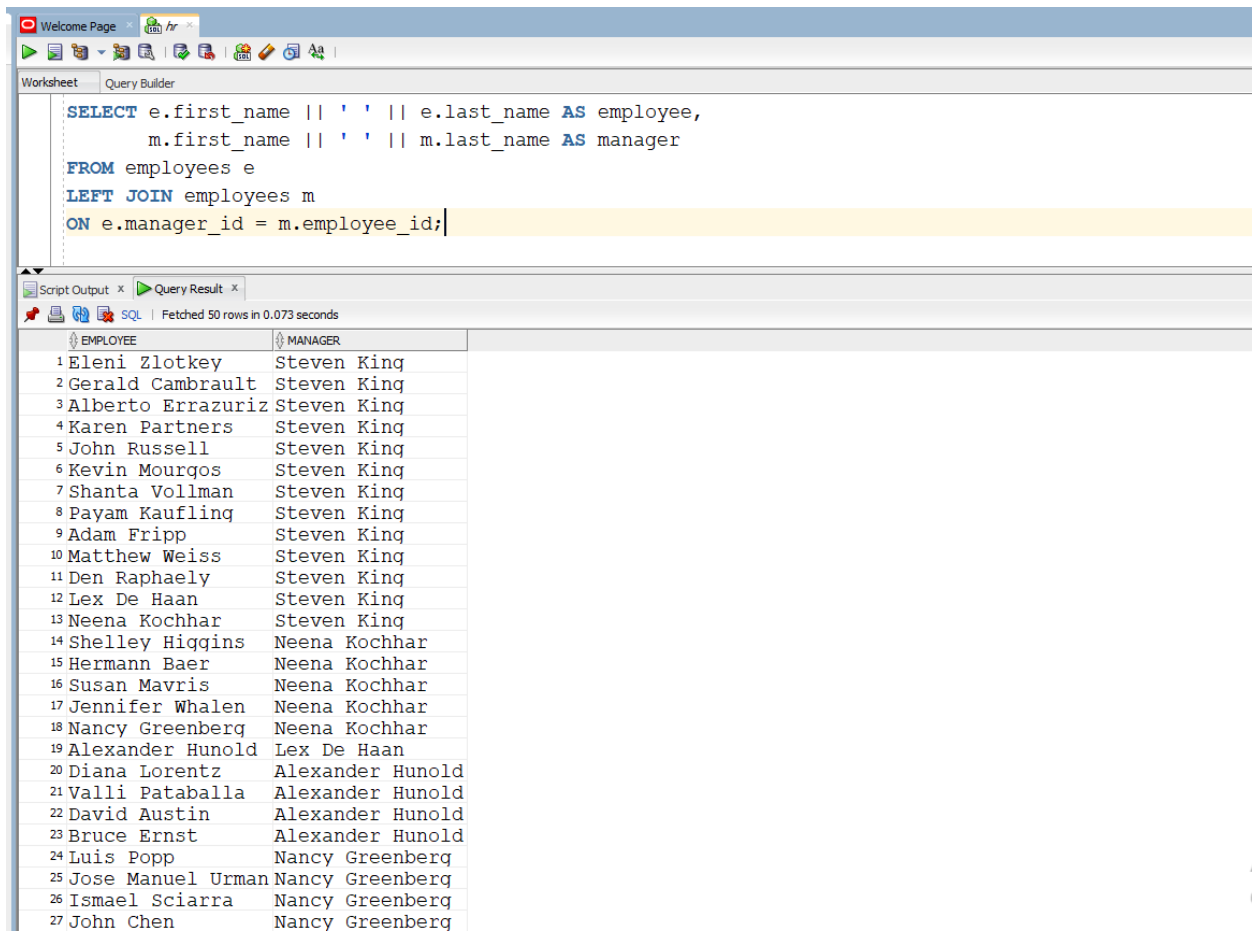


The screenshot shows a database query tool interface. The top section is the 'Query Builder' tab, which contains a SQL query. The query is a nested SELECT statement that filters employees based on the number of employees in each department. The bottom section is the 'Query Result' tab, which displays the results of the query in a table format. The table has two columns: 'FIRST_NAME' and 'LAST_NAME'. The results list 23 employees, each with their first and last name.

```
SELECT first_name, last_name
FROM employees
WHERE department_id IN (
  SELECT department_id
  FROM employees
  GROUP BY department_id
  HAVING COUNT(*) > 5
);
```

	FIRST_NAME	LAST_NAME
1	Laura	Bissot
2	Mozhe	Atkinson
3	James	Marlow
4	John	Seo
5	Allan	McEwen
6	Clara	Vishney
7	Danielle	Greene
8	Harrison	Bloom
9	William	Smith
10	Anthony	Cabrio
11	Sarah	Bell
12	Shelli	Baida
13	Guy	Himuro
14	Irene	Mikkilineni
15	Alberto	Errazuriz
16	Oliver	Tuvault
17	Janette	King
18	Douglas	Grant
19	Jason	Mallin
20	Randall	Matos
21	Elizabeth	Bates
22	Nandita	Sarchand
23	Randall	Perkins

TASK 4:



The screenshot shows a SQL query builder interface. The top section displays the query:

```
SELECT e.first_name || ' ' || e.last_name AS employee,  
       m.first_name || ' ' || m.last_name AS manager  
FROM employees e  
LEFT JOIN employees m  
ON e.manager_id = m.employee_id;
```

The bottom section shows the query results, which are 50 rows fetched in 0.073 seconds. The results are displayed in a table with two columns: EMPLOYEE and MANAGER.

	EMPLOYEE	MANAGER
1	Eleni Zlotkey	Steven King
2	Gerald Cambrault	Steven King
3	Alberto Errazuriz	Steven King
4	Karen Partners	Steven King
5	John Russell	Steven King
6	Kevin Mourgou	Steven King
7	Shanta Vollman	Steven King
8	Payam Kaufling	Steven King
9	Adam Fripp	Steven King
10	Matthew Weiss	Steven King
11	Den Raphaely	Steven King
12	Lex De Haan	Steven King
13	Neena Kochhar	Steven King
14	Shelley Higgins	Neena Kochhar
15	Hermann Baer	Neena Kochhar
16	Susan Mavris	Neena Kochhar
17	Jennifer Whalen	Neena Kochhar
18	Nancy Greenberg	Neena Kochhar
19	Alexander Hunold	Lex De Haan
20	Diana Lorentz	Alexander Hunold
21	Valli Pataballa	Alexander Hunold
22	David Austin	Alexander Hunold
23	Bruce Ernst	Alexander Hunold
24	Luis Popp	Nancy Greenberg
25	Jose Manuel Urman	Nancy Greenberg
26	Ismael Sciarra	Nancy Greenberg
27	John Chen	Nancy Greenberg

TASK 5:

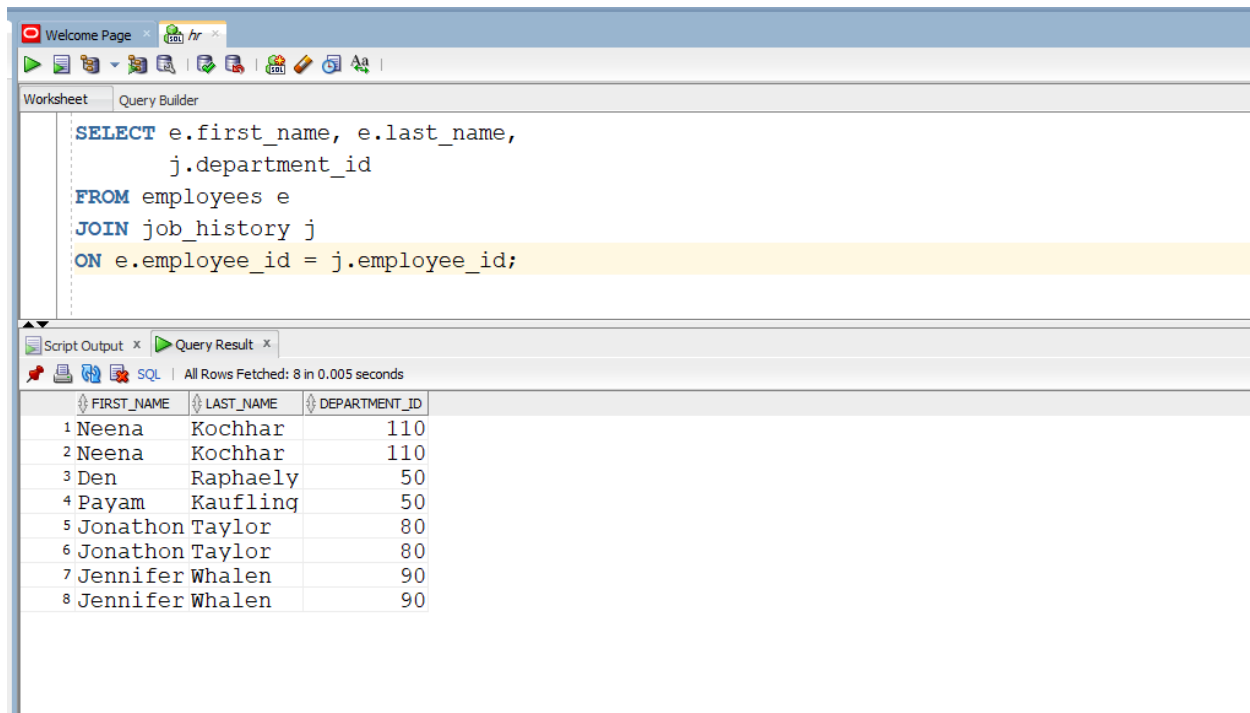
The screenshot shows the Oracle SQL Developer interface. The top pane displays a SQL query in the Query Builder. The query is designed to find employees whose salary is greater than the average salary of their department but less than the maximum salary of the entire company.

```
SELECT first_name, last_name, salary
FROM employees e
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department_id = e.department_id
)
AND salary < (
    SELECT MAX(salary)
    FROM employees
);
```

The bottom pane shows the Query Result, which contains 20 rows of data. The status bar indicates that all rows were fetched in 0.032 seconds.

	FIRST_NAME	LAST_NAME	SALARY
1	Daniel	Faviet	9000
2	Nancy	Greenberg	12008
3	Den	Raphaely	11000
4	Shelley	Higgins	12008
5	Britney	Everett	3900
6	Sarah	Bell	4000
7	Jennifer	Dilly	3600
8	Kelly	Chung	3800
9	Alexis	Bull	4100
10	Nandita	Sarchand	4200
11	Trenna	Rajs	3500
12	Renske	Ladwig	3600
13	Kevin	Mourgos	5800
14	Shanta	Vollman	6500
15	Payam	Kaufling	7900
16	Adam	Fripp	8200
17	Matthew	Weiss	8000
18	Ellen	Abel	11000
19	Tayler	Fox	9600
20	Harrison	Bloom	10000

TASK 6:



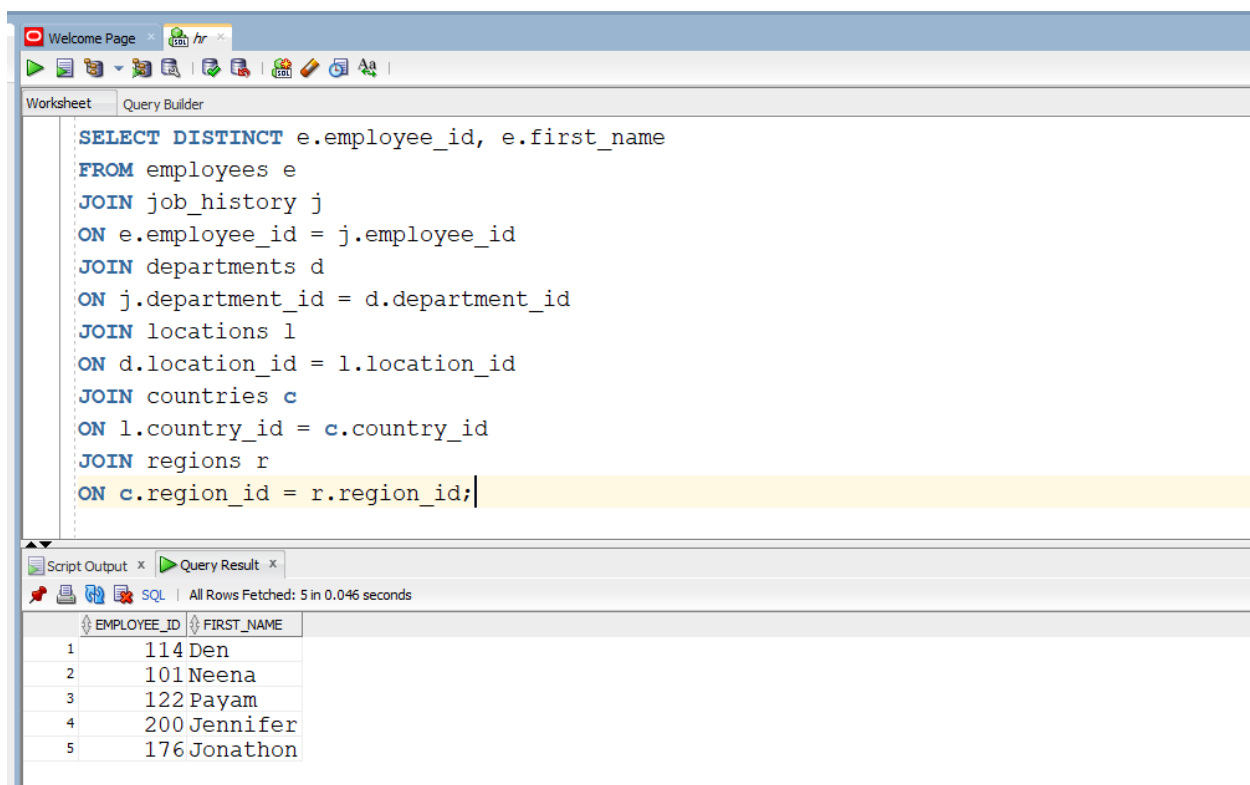
The screenshot shows the SQL Developer interface. The Query Builder pane contains the following SQL query:

```
SELECT e.first_name, e.last_name,  
       j.department_id  
FROM employees e  
JOIN job_history j  
ON e.employee_id = j.employee_id;
```

The Query Result pane shows the results of the query, with 8 rows fetched in 0.005 seconds. The results are displayed in a table with columns FIRST_NAME, LAST_NAME, and DEPARTMENT_ID.

	FIRST_NAME	LAST_NAME	DEPARTMENT_ID
1	Neena	Kochhar	110
2	Neena	Kochhar	110
3	Den	Raphaely	50
4	Payam	Kaufling	50
5	Jonathon	Taylor	80
6	Jonathon	Taylor	80
7	Jennifer	Whalen	90
8	Jennifer	Whalen	90

TASK 7:



The screenshot shows the SQL Developer interface. The Query Builder pane contains the following SQL query:

```
SELECT DISTINCT e.employee_id, e.first_name  
FROM employees e  
JOIN job_history j  
ON e.employee_id = j.employee_id  
JOIN departments d  
ON j.department_id = d.department_id  
JOIN locations l  
ON d.location_id = l.location_id  
JOIN countries c  
ON l.country_id = c.country_id  
JOIN regions r  
ON c.region_id = r.region_id;
```

The Query Result pane shows the results of the query, with 5 rows fetched in 0.046 seconds. The results are displayed in a table with columns EMPLOYEE_ID and FIRST_NAME.

	EMPLOYEE_ID	FIRST_NAME
1	114	Den
2	101	Neena
3	122	Payam
4	200	Jennifer
5	176	Jonathon

TASK 8:

The screenshot shows a SQL query editor interface. The top toolbar includes icons for running queries, saving, and other standard database tools. The main text area contains the following SQL query:

```
SELECT e1.last_name
FROM employees e1
JOIN employees e2
ON e1.last_name = e2.last_name
AND e1.department_id <> e2.department_id;
```

Below the query editor, the 'Query Result' tab is active, displaying the results of the query. The results are shown in a table with a single column labeled 'LAST_NAME'.

	LAST_NAME
1	King
2	King
3	Taylor
4	Taylor

The status bar at the bottom indicates 'All Rows Fetched: 4 in 0.006 seconds'.

TASK 9:

Welcome Page

Worksheet Query Builder

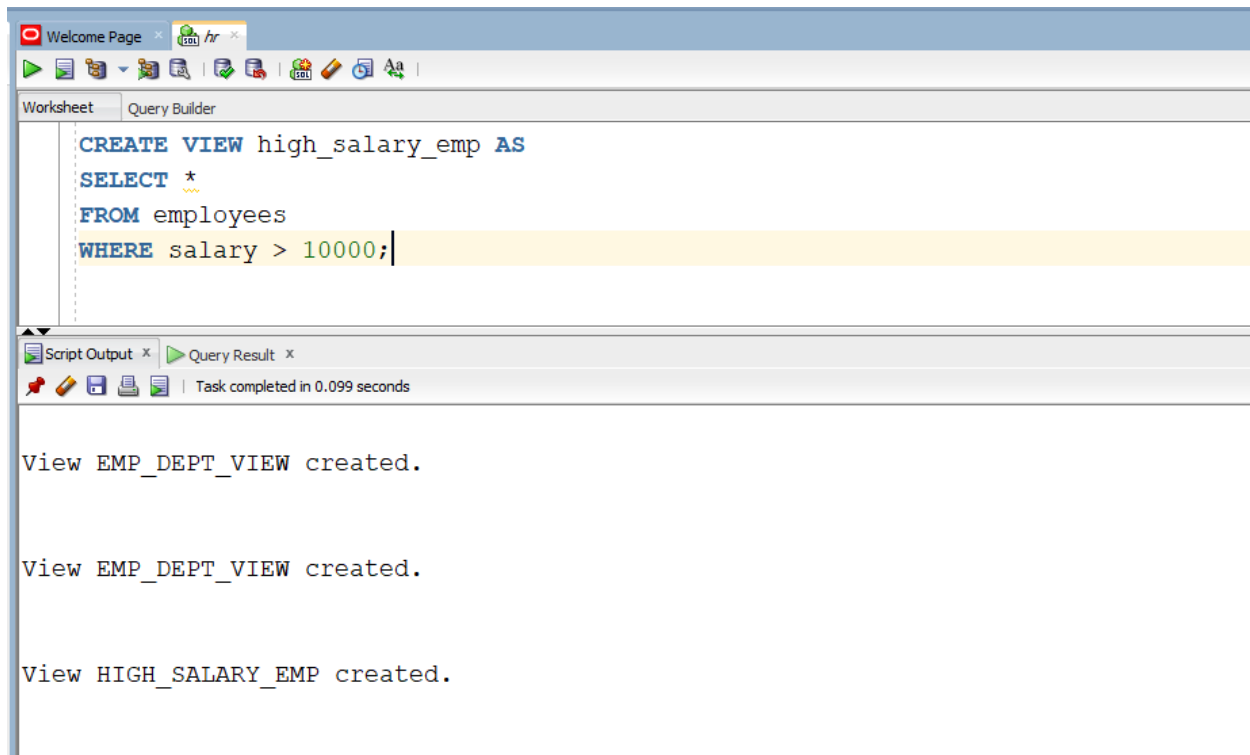
```
SELECT *  
FROM (  
    SELECT first_name, last_name, salary, department_id,  
           RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) r  
    FROM employees  
)  
WHERE r <= 3;
```

Script Output x Query Result x

SQL | All Rows Fetched: 25 in 0.01 seconds

	FIRST_NAME	LAST_NAME	SALARY	DEPARTMENT_ID	R
1	Jennifer	Whalen	4400	10	1
2	Den	Raphaely	11000	30	1
3	Alexander	Khoo	3100	30	2
4	Shelli	Baida	2900	30	3
5	Susan	Mavris	6500	40	1
6	Adam	Fripp	8200	50	1
7	Matthew	Weiss	8000	50	2
8	Payam	Kaufling	7900	50	3
9	Alexander	Hunold	9000	60	1
10	Bruce	Ernst	6000	60	2
11	David	Austin	4800	60	3
12	Valli	Pataballa	4800	60	3
13	Hermann	Baer	10000	70	1
14	John	Russell	14000	80	1
15	Karen	Partners	13500	80	2
16	Alberto	Errazuriz	12000	80	3
17	Steven	King	24000	90	1
18	Neena	Kochhar	17000	90	2
19	Lex	De Haan	17000	90	2
20	Nancy	Greenberg	12008	100	1
21	Daniel	Faviet	9000	100	2
22	John	Chen	8200	100	3
23	Shelley	Higgins	12008	110	1
24	William	Gietz	8300	110	2

TASK 10:



The screenshot shows the SQL Developer interface. The top toolbar includes icons for running queries, saving, and other database functions. The 'Query Builder' tab is active, displaying the following SQL statement:

```
CREATE VIEW high_salary_emp AS
SELECT *
FROM employees
WHERE salary > 10000;
```

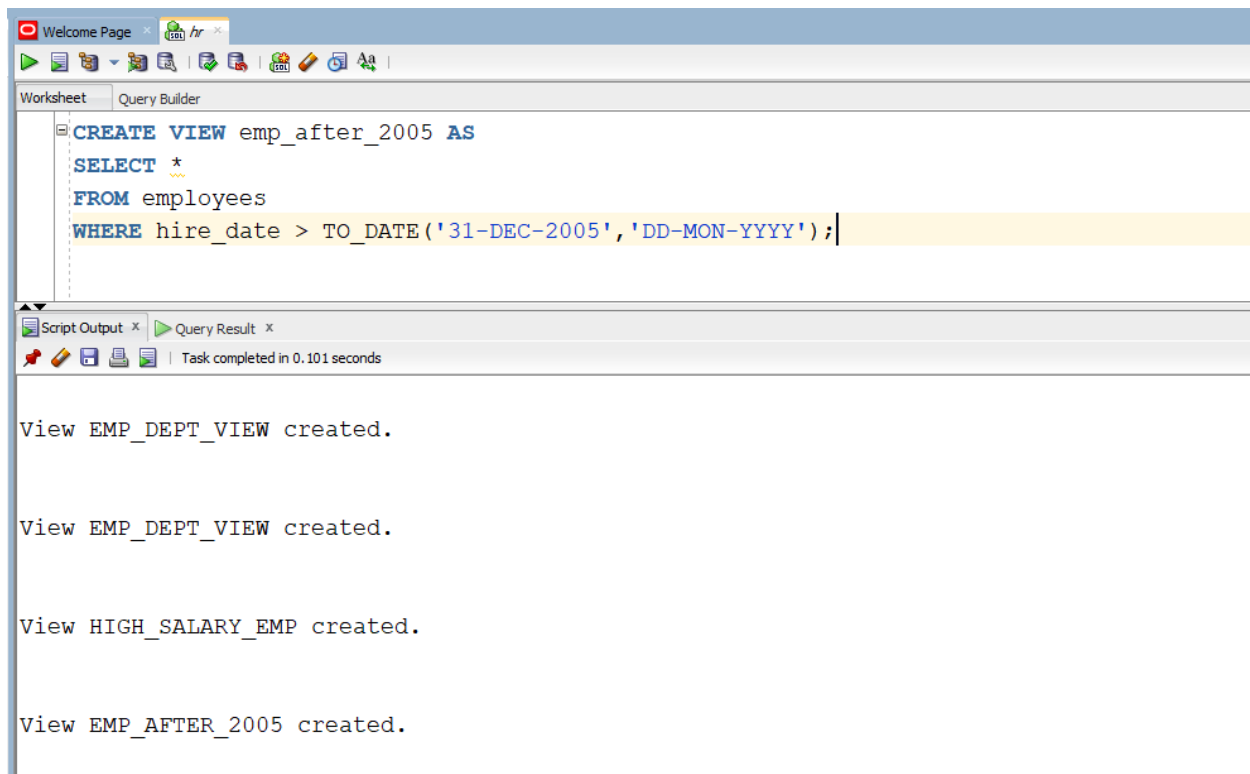
Below the query editor, the 'Script Output' and 'Query Result' tabs are visible. The 'Script Output' tab shows the following messages:

```
View EMP_DEPT_VIEW created.

View EMP_DEPT_VIEW created.

View HIGH_SALARY_EMP created.
```

TASK 11:



The screenshot shows the SQL Developer interface. The top toolbar includes icons for running queries, saving, and other database functions. The 'Query Builder' tab is active, displaying the following SQL statement:

```
CREATE VIEW emp_after_2005 AS
SELECT *
FROM employees
WHERE hire_date > TO_DATE('31-DEC-2005', 'DD-MON-YYYY');
```

Below the query editor, the 'Script Output' and 'Query Result' tabs are visible. The 'Script Output' tab shows the following messages:

```
View EMP_DEPT_VIEW created.

View EMP_DEPT_VIEW created.

View HIGH_SALARY_EMP created.

View EMP_AFTER_2005 created.
```